

SORTING - 2

"Motivation is what gets you started. Habit is what keeps you going."

~ Jim Ryun



THE BRIGHT
DROPS
.com



Good
Evening ☺

Content for Today

01. Inversion count
02. Inbuilt sorting function
03. Custom comparison
04. Factors sort
05. Largest number
06. Custom comparator on class

Inversion count

Given an integer array, count the number of inversion pairs in the array

Inversion pair $\rightarrow (i, j)$ where $(i < j \ \&\& \ A[i] > A[j])$

$A = [4 \ 5 \ 1 \ 2 \ 6 \ 3]$
0 1 2 3 4 5

No. of inversion pair = 7

$(0, 2) \ (0, 3) \ (0, 5)$

$(1, 2) \ (1, 3) \ (1, 5)$

$(4, 5)$

$A = [8, 3, 4]$
0 1 2

No. of inversion pair = 2

$(0, 1) \ (0, 2)$

Brute force $\rightarrow$ check for all the pairs (i, j) & $i < j \ \&\& \ A[i] > A[j]$, increase the count

TC: $O(n^2)$

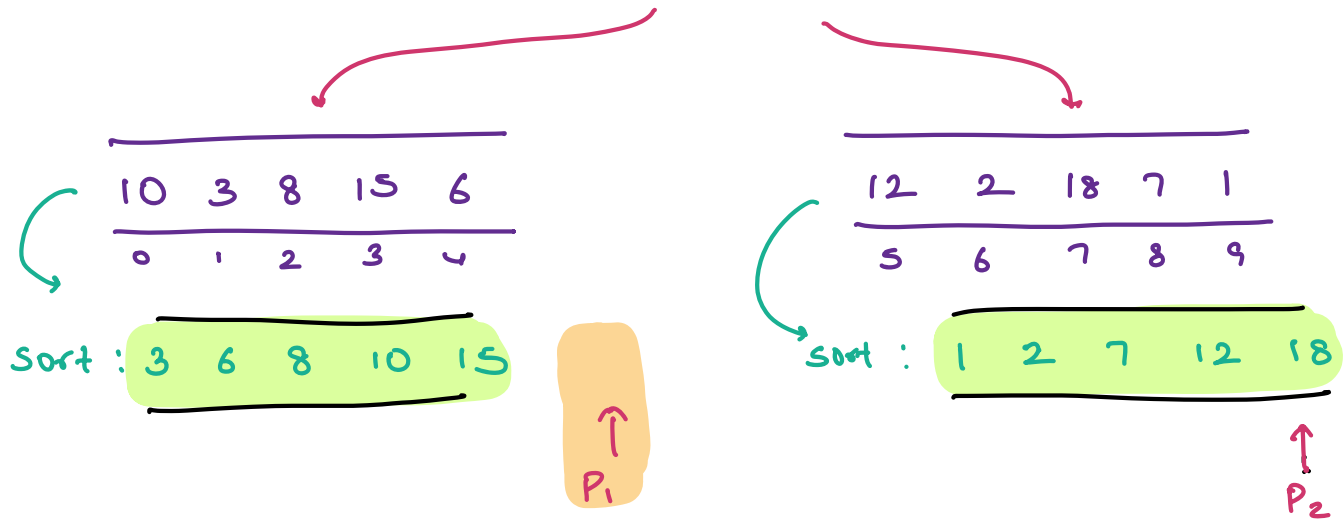
SC: $O(1)$

Ans = $6 + 2 + 4 + 5 + 2 + 3 + 1 + 2 + 1 + 0 = 26$

Idea 2 $\rightarrow$ **To use merge sort**

arr[] = { 10 3 8 15 6 12 2 18 7 1 }

0 1 2 3 4 5 6 7 8 9



Merged: 1 | 2 | 3 | 6 | 7 | 8 | 10 | 12 | 15 | 18

ans = 5 + 5 + 3 + 1 $\Rightarrow$ 14 inversion pairs

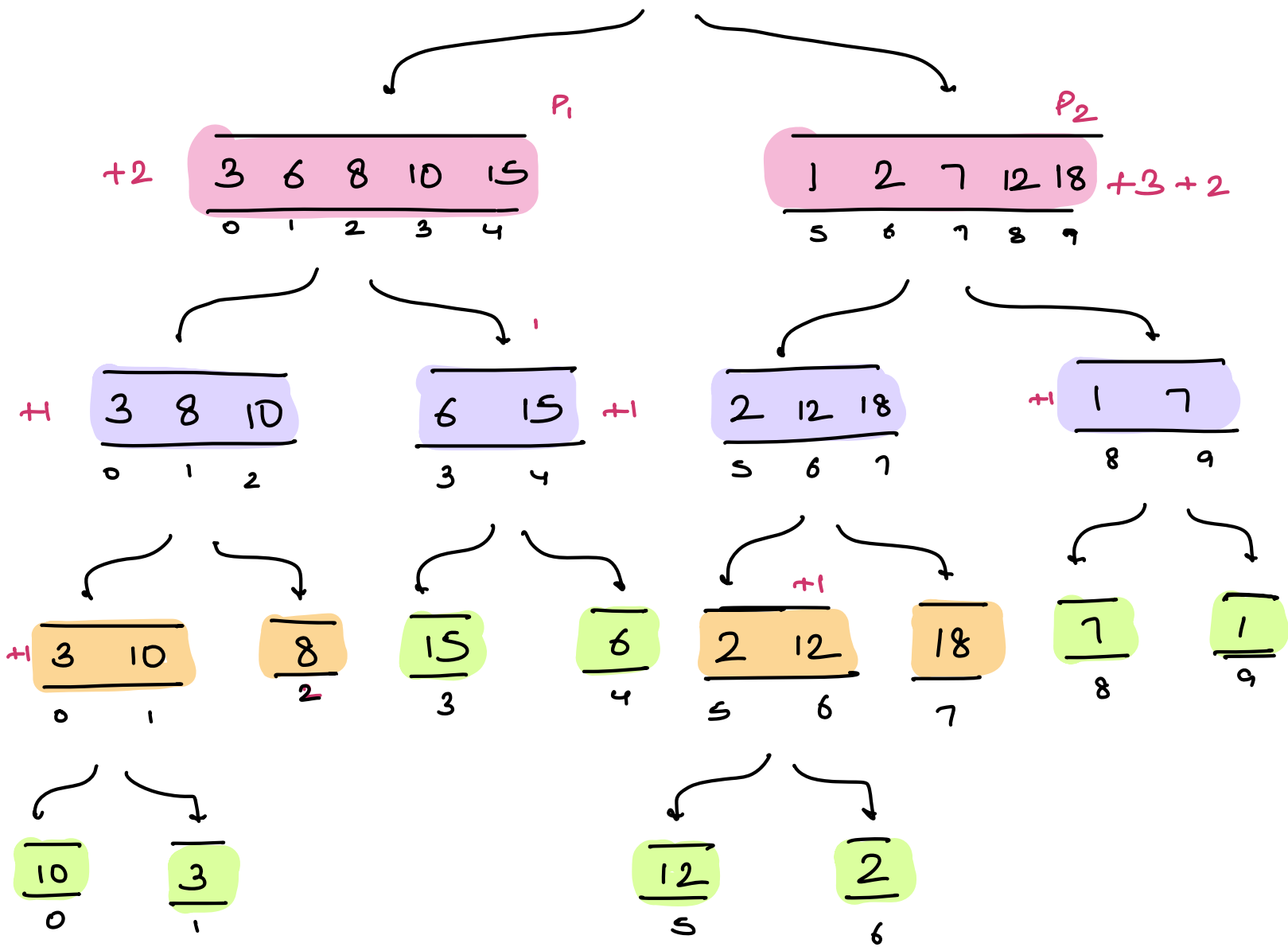
$a[P_1] < a[P_2] \rightarrow$ insert

$a[P_1] > a[P_2] \rightarrow$ inversion pairs

Also consider the inversion pairs present in only left part & inversion pairs only in right part.

26 inversion

arr[] = { 1 2 3 6 7 8 10 12 15 18 } +5+5+3
 0 1 2 3 4 5 6 7 8 9 +1



$P_1 < P_2$ & if ($arr[P_1] > arr[P_2]$) } $\rightarrow$ Inversion pair

mergesort (int [] ar, int s, int e)

if (s == e) return;

m = (s + e) / 2;

mergesort (ar, s, m) $\rightarrow f(n/2)$

mergesort (ar, m+1, e) $\rightarrow f(n/2)$

merge (ar, s, m, e) $\rightarrow O(n)$

3

TC: $O(n \log n)$

static int ans = 0

void merge (int a[n], int s, m, e)

int [] c = new int [e - s + 1]

P₁ = s, P₂ = m+1, k = 0

while (P₁ ≤ m && P₂ ≤ e)

if (a[P₁] < a[P₂])

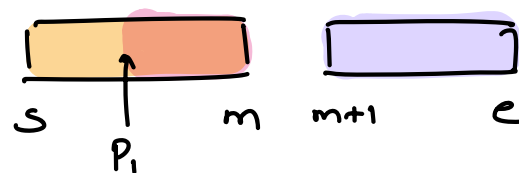
c[k] = a[P₁]

P₁ ++;

k ++;

3

else if (a[P₁] > a[P₂]) {



count of ele

↳ m - P₁ + 1

```
c[k] = a[p2]
```

```
p2++;
```

```
k++;
```

```
ans = ans + m - p1 + 1 ;
```

```
while (p1 ≤ m)
```

```
c[k] = a[p1]
```

```
p1++;
```

```
k++;
```

```
while (p2 ≤ e)
```

```
c[k] = a[p2]
```

```
p2++;
```

```
k++;
```

```
for (i = 0 ; i < c.length ; i++)
```

```
    ar[i+s] = c[i]
```

* **Sorting function** → Inbuilt function

$\left\{ \begin{array}{l} \rightarrow \text{Collections.sort}(al) \\ \rightarrow \text{Arrays.sort}(arr) \end{array} \right\}$

$\xrightarrow{\text{Arraylist}}$

$\xrightarrow{\text{Array}}$

$\left. \begin{array}{l} \text{TC: } O(n \log n) \\ \text{SC: } O(1) \end{array} \right\}$

Merge sort + Quick sort

$\underbrace{\hspace{10em}}$

Tim sort // check on google

* **Array of Integers**

$A[] = \{10, 3, 7, 18\}$

$\text{Arrays.sort}(A)$: $\xrightarrow{\text{name of array}}$

Output = $A[] = \{3, 7, 10, 18\}$

// Ascending order is the default behaviour

* Array of characters

```
char [] ch = {'a', 'k', 'e', 'c'};
```

`Arrays.sort(ch);` → basis of ASCII value

Output :- `char [] ch : { a, c, e, k };`

* Array of String

```
String [] str = { "bat", "add", "apple" };
```

`Arrays.sort(str);` // → sort lexicographically or alphabetical order

Output : `{ "add", "apple", "bat" };`

* Custom comparison

Q - Given an array of `String []`. Sort the array on the basis on length of string

String [] s = { "char", "it", "bed", "carrot" }

④

②

③

⑥

Output [] s = { "it", "bed", "char", "carrot" }

Q = Arrange integers in descending order

arr [] = { -10, 3, 8, 2 }

Output = { 8, 3, 2, -10 }

* Custom comparison

↳ Using an interface known as

comparator to provide the

parameter on the basis we have
to sort

* Default behaviour of sort function

Ascending order

-ve $\rightarrow$ move it on LHS

0 $\rightarrow$ neutral value

+ve $\rightarrow$ move it on RHS



* Sort the array in descending order

Look for pinned msg.

Solved it on IDE

Factors Sort

Q Given N elements, sort them in increasing order of their no. of factors. If 2 elements have same no. of factor, elements with less value should come first.

Note:- {No Extra space}

arr[] = { 9 3 4 8 16 37 6 13 15 }

0 1 2 3 4 5 6 7 8

No. of factors = [3] [2] [3] [4] [5] [2] [4] [2] [4]

Output → { 3, 13, 37, 4, 9, 6, 8, 15, 16 }

Quiz → arr[] = { 12, 5, 10, 4, 7, 6, 9, 1, 3 } ;

No. of factors = 6 2 4 3 2 4 3 1 2

Output = { 1, 3, 5, 7, 4, 9, 6, 10, 12 }

Solved on IDE

03. Largest Number { v. v. v. v. 1 }

Given an array of integers, arrange them in such a way that they form the largest number.

Note:- The result may be very large, so return a string instead of an integer

Constraints

$$1 \leq n \leq 10^5$$

$$0 \leq A[i] \leq 10^8$$

$$A = \{2, 3, 9, 0\} \Rightarrow 9320$$

$$A = \{88, 9\} = 988$$

$$A = \{30, 3\} = 330$$



* Create the numbers

* Sort them according to the value

* Real world implementation $\Rightarrow$ Student {
| Name
| roll no
| marks
3